Server

1. Create a server socket and bind it to the port.
2. Listen for new connections and when a connection arrives, accept it.
3. Send server's date and time to the client.
4. Read the client's IP address sent by the client.
5. Display the client details.
6. Repeat steps 2-5 until the server is terminated.
7. Close all streams.
8. Close the server socket.
9. Stop.

Client

1. Create a client socket and connect it to the server's port number.
2. Retrieve its own IP address using built-in function.
3. Send its address to the server.
4. Display the date & time sent by the server.
5. Close the input and output streams.
6. Close the client socket.
7. Stop.

Server_DT

```
import java.net.*;
import java.io.*;
import java.util.Date;
public class Server_DT {

   public static void main(String[] args)throws IOException {
      // TODO code application logic here

      //Step 1. Reserve a port number on the Server to offer this service
      ServerSocket ss= new ServerSocket(5000);
      //(Optional)To confirm Server Reserved specified port or not
      System.out.println("The Server has reserved port No.: "+ss.getLocalPort()+" for this Service");

      //Step 2. Now create a Client Socket on Server for Bidirectonal Communication.
      //Socket is created only when client communicates with the server

         Socket cs=ss.accept();

         //To confirm Server communicated through the socket or not
          System.out.println("Client with IP Address "+cs.getInetAddress()+" has communicated via port
                No.: "+cs.getPort());

         Date d=new Date();
         String s="Current Date & Time on Server is:"+d;

         //Send String s to client via client socket
         PrintWriter toclient=new PrintWriter(cs.getOutputStream(),true);
         toclient.print(s);
         toclient.close();
         cs.close();
         ss.close();
        }

}
```

Client_DT

```
import java.net.*;
import java.io.*;

public class Client_DT {

    public static void main(String[] args) throws UnknownHostException,IOException {
        // TODO code application logic here
        //Step 1. Create a client socket to connect to Server
        Socket cs= new Socket("LocalHost",5000);

        //To confirm Client is communicating through the port
        System.out.println("Client "+cs.getInetAddress()+" is communicating from port No.:
                "+cs.getPort());

        //Receive Date Sent by Server

BufferedReader fromserver=new BufferedReader(new InputStreamReader(cs.getInputStream()));

System.out.println(fromserver.readLine());
fromserver.close();
cs.close();

    }
}
```

These steps outline the key tasks involved in creating a concurrent day-time server in Java, handling client connections, sending the date and time, and properly managing resources. Here's an explanation of each step:

1. Create a New Server and Bind It to a Port

- **Explanation:**

  - The server needs to be created and bound to a specific port number so that clients know where to connect. This involves creating a `ServerSocket` object in Java, which listens for incoming client connections.

  - **Code Example:**

    ```java

    int port = 5000; // Define the port number

    ServerSocket serverSocket = new ServerSocket(port); // Create and bind server to the port

    ```

2. Listen to New Connections and Accept Them

- **Explanation:**

  - After binding the server to a port, the server should continuously listen for incoming connection requests from clients. When a client connects, the server accepts the connection.

  - **Code Example:**

    ```java

    Socket clientSocket = serverSocket.accept(); // Listen and accept a new client connection

    ```

  - This step blocks the server until a client connects.

3. Send the Server's Date and Time to the Client

- **Explanation:**

  - Once the server accepts a client connection, it should send the current date and time to the client. This is done by writing the date and time to the output stream associated with the client socket.

  - **Code Example:**

    ```java

    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

    String dateTime = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new Date());
    ```

```
out.println("Current date and time: " + dateTime);
```

4. Read the Client's IP Address Sent by the Client

- **Explanation:**

  - The server can retrieve the client's IP address directly from the socket. This IP address is part of the connection metadata and is not sent explicitly by the client.

- **Code Example:**

  ```java
  String clientIP = clientSocket.getInetAddress().getHostAddress();
  ```

5. Display the Client Details

- **Explanation:**

  - After reading the client's IP address, the server should display the client's details, including the IP address and possibly other information, to the console for logging purposes.

- **Code Example:**

  ```java
  System.out.println("Client connected: IP Address = " + clientIP);
  ```

6. Repeat Steps 2-5 Until the Server is Terminated

- **Explanation:**

  - The server needs to continue listening for new client connections and serving them one by one. This is typically done inside a loop that runs indefinitely until the server is manually stopped.

- **Code Example:**

  ```java
  while (true) {
      Socket clientSocket = serverSocket.accept(); // Step 2
```

```
        // Steps 3, 4, 5 would follow here...
    }
    ```

7. Close All Streams

- **Explanation:**

  - After serving a client, the server should close all associated input and output streams to free up resources. This is crucial to avoid resource leaks.

  - **Code Example:**

    ```java
    out.close(); // Close the output stream
    clientSocket.close(); // Close the client socket
    ```

8. Close the Server Socket

- **Explanation:**

  - When the server is shutting down (e.g., after an admin stops it), it should close the `ServerSocket` to release the port and stop accepting new connections.

  - **Code Example:**

    ```java
    serverSocket.close(); // Close the server socket
    ```

9. Stop

- **Explanation:**

  - This step involves the graceful termination of the server. After closing the server socket and ensuring that all resources are released, the server stops running.

  - This might involve breaking out of the loop and performing any necessary cleanup.

Example Code Combining These Steps

Here's a simplified example that puts these steps together in a basic server implementation:

```java
import java.io.*;
import java.net.*;
import java.text.SimpleDateFormat;
import java.util.Date;

public class DayTimeServer {

    public static void main(String[] args) {
        int port = 5000; // Step 1

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server started on port " + port);

            while (true) { // Step 6
                // Step 2
                Socket clientSocket = serverSocket.accept();

                // Step 4: Get client's IP address
                String clientIP = clientSocket.getInetAddress().getHostAddress();

                // Step 5: Display client details
                System.out.println("Client connected: IP Address = " + clientIP);
```

```
            // Step 3: Send date and time to the client
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
            String dateTime = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new
Date());
            out.println("Current date and time: " + dateTime);

            // Step 7: Close all streams
            out.close();
            clientSocket.close();
        }
    } catch (IOException e) {
        System.err.println("Server exception: " + e.getMessage());
    } finally {
        // Step 8: Close the server socket
        try {
            if (serverSocket != null && !serverSocket.isClosed()) {
                serverSocket.close();
            }
        } catch (IOException e) {
            System.err.println("Failed to close server socket: " + e.getMessage());
        }
    }

    // Step 9: Stop (handled by breaking out of the loop and closing resources)
    System.out.println("Server stopped.");
  }
}
```

This code creates a server that listens on port 5000, accepts client connections, sends the current date and time, and displays client details. The server runs indefinitely until it is manually stopped.

To implement the client-side of a concurrent day-time client-server application in Java, you can follow these steps. Each step involves creating the necessary functionality to connect to the server, exchange information, and then properly manage the resources.

1. Create a Client Socket and Connect It to the Server's Port

- **Explanation:**

  - The client must create a `Socket` object that connects to the server using the server's hostname (or IP address) and the port number where the server is listening.

  - **Code Example:**

    ```java

    String serverHost = "localhost"; // Server's hostname or IP address

    int serverPort = 5000; // Server's port number

    Socket clientSocket = new Socket(serverHost, serverPort); // Create and connect the socket

    ```

2. Receive Its Own IP Address

- **Explanation:**

  - After the client is connected, it can retrieve its own IP address from the socket. This is the local IP address assigned to the client during the connection.

  - **Code Example:**

    ```java

    String clientIP = clientSocket.getLocalAddress().getHostAddress(); // Get client's own IP address

    ```

3. Send the Address to the Server

- **Explanation:**

- The client should send its IP address to the server using the output stream associated with the socket. This allows the server to log or process the client's IP address if needed.

- **Code Example:**

  ```java
  PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
  out.println(clientIP); // Send client's IP address to the server
  ```

4. Display the Date and Time Sent by the Server

- **Explanation:**

  - After sending its IP address, the client should listen for the server's response, which will contain the current date and time. The client then displays this information.

- **Code Example:**

  ```java
  BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
  String serverResponse = in.readLine(); // Read the date and time sent by the server
  System.out.println("Server response: " + serverResponse); // Display the server's response
  ```

5. Close All Input and Output Streams

- **Explanation:**

  - Once the interaction with the server is complete, the client should close its input and output streams to free up resources.

- **Code Example:**

  ```java
  in.close(); // Close the input stream
  out.close(); // Close the output stream
  ```

6. Close the Client Socket

- **Explanation:**

  - After closing the streams, the client should close the socket to terminate the connection with the server.

  - **Code Example:**

    ```java

    clientSocket.close(); // Close the client socket

    ```

7. Stop

- **Explanation:**

  - The client program ends after closing the socket and streams, completing its operation.

Example Code Combining These Steps

Here's a complete Java client-side program that implements these steps:

```java
import java.io.*;
import java.net.*;

public class DayTimeClient {

    public static void main(String[] args) {
        String serverHost = "localhost"; // Server's hostname or IP address
        int serverPort = 5000; // Server's port number

        try (Socket clientSocket = new Socket(serverHost, serverPort)) {
            // Step 2: Get the client's own IP address
```

```
            String clientIP = clientSocket.getLocalAddress().getHostAddress();

            // Step 3: Send the client's IP address to the server
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
            out.println(clientIP);

            // Step 4: Display the date and time sent by the server
            BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
            String serverResponse = in.readLine();
            System.out.println("Server response: " + serverResponse);

            // Step 5: Close all input and output streams
            in.close();
            out.close();

            // Step 6: Close the client socket
            clientSocket.close();

        } catch (IOException e) {
            System.err.println("Client exception: " + e.getMessage());
        }

        // Step 7: Stop
        System.out.println("Client stopped.");
    }
}
```

How the Client Works:

1. **Create and Connect the Socket:** The client connects to the server using the server's hostname and port.

2. **Get and Send IP Address:** The client retrieves its own IP address and sends it to the server.

3. **Receive and Display Date/Time:** The client receives the date and time from the server and prints it out.

4. **Clean Up Resources:** The client closes its input and output streams, then closes the socket, and the program terminates.

This client can connect to the server described in the previous sections, send its IP address, receive the current date and time, and then properly close the connection, ensuring a clean shutdown.